

Algorithm for file updates in Python

Project description

This portfolio activity for Course 7 of Google Cybersecurity Certificate focuses on the python algorithm to access the “allow_list.txt” file and remove IP addresses so that they don’t have access.

Open the file that contains the allow list

```
import_file = “allow_list.txt”
```

The code above can be read like this:

import_file = assigned variable
“allow_list.txt” = name of the file and file format

Therefore, it is stated that you’re assigning “allow_list.txt” to var import_file.

```
With open(import_file, “r”) as file:
```

The code above can be read like this:

open () = function on what to do with the file

- import_file = first parameter you import
- “r” = the second parameter on what you want to do with the file. It can also be “w” for write and much more.

as file = assign a variable named file to store the output of “import_file”

Read the file contents

```
With open(import_file, “r”) as file:
```

```
ip_addresses = file.read()
```

This command can be read like this:

- You have an assigned variable called ip_addresses where you have a method called .read() to read the contents of the file [hence file.read()] in string (for this instance, it is import_file).

Convert the string into a list

```
ip_addresses = ip_addresses.split()
```

This command can be read like this:

- You have an assigned variable called `ip_addresses` where you have a method called `.split()` to convert “`ip_addresses`” from string in list type of the file [hence `ip_addresses.split()`].

Iterate through the remove list

```
for element in remove_list:
```

This command can be interpreted below:

- for loop (repeats code based from the sequence implemented until completion).
The next sequence is the element (variable inside the loop); in (iterative sequence);
and `remove_list` (sequence of `ip_addresses`) assigned to the loop variable element.

Remove IP addresses that are on the remove list

```
for element in remove_list:
```

```
    if element in ip_addresses:
```

```
        ip_addresses.remove(element)
```

The code can be interpreted like this:

- Once you have the for loop implemented, created an if statement that evaluate whether the element (variable in if statement) found inside `ip_addresses` list. Once that is done, created inside the if statement the `ip_addresses.remove(element)` so that IP addresses found in the `remove_list` are removed from `ip_addresses`.

Update the file with the revised list of IP addresses

```
ip_addresses = "\n".join(ip_addresses)
```

```
with open(import_file, "w") as file:
```

```
    file.write(ip_addresses)
```

The code above can be interpreted like this:

- First code: `ip_addresses = "\n".join(ip_addresses)` = `.join()` method that combines all iterable items into a string. So `ip_addresses` has an assigned value “`\n`” (place element on a new line), then lastly `.join(ip_addresses)`.

- Second code: with open(import_file, "w") as file that opens and writes the import_file. File.write(ip_addresses) to rewrite the file and its contents with ip_addresses.

Summary

The overall summary here is the utilization of python functions and methods. From for loop to .write(). These are useful for a python coding algorithm focused on removing lists of IP addresses and write over a new "allow_list.txt" with the revised list of IP addresses.